

ICC4200 - Sistemas Embebidos 202610

Laboratorio 3: TinyML

Ejercicio 1.1: Implementación directa

Integrantes:

Beatriz Bretti

Simón Alonso

Julian Pineda

Matias García-Huidobro

Fecha: 27 de mayo de 2026

1. Objetivo del ejercicio

El objetivo de este ejercicio fue comparar la ejecución del modelo *hello world* de TensorFlow Lite Micro con una implementación directa en C. El modelo original se ejecutó sobre una placa ESP32-S3 utilizando ESP-IDF y la librería `esp-tflite-micro`. Posteriormente, se implementó una versión directa de la función aproximada por el modelo, utilizando código C/C++ sin pasar por el intérprete de TensorFlow Lite Micro.

La comparación se realizó considerando dos aspectos principales: la salida generada por ambos métodos y el tiempo de inferencia.

2. Modelo utilizado

El ejemplo *hello world* de TensorFlow Lite Micro corresponde a un modelo pequeño entrenado para aproximar una función sinusoidal. En este caso, el modelo recibe como entrada un valor escalar x y genera como salida un valor y que aproxima el comportamiento de una función seno.

La topología del modelo fue revisada utilizando Netron. Como se observa en la Figura 1, el modelo está compuesto por tres capas *Fully Connected*. La entrada corresponde a un único valor. Las dos primeras capas tienen 16 neuronas cada una y utilizan activación ReLU. La última capa entrega un único valor de salida.

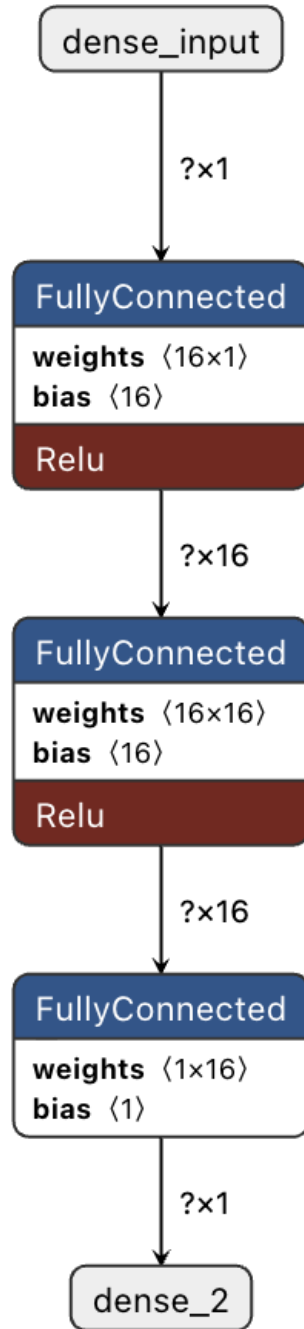


Figura 1: Topología del modelo *hello world* visualizada en Netron.

3. Implementación en TensorFlow Lite Micro

Primero se ejecutó el ejemplo original utilizando TensorFlow Lite Micro. En este caso, el programa carga el modelo cuantizado, asigna memoria mediante `tensor_arena`, prepara el tensor de entrada, ejecuta la inferencia con `interpreter->Invoke()` y finalmente obtiene la salida desde el tensor de salida.

El flujo general de inferencia con TensorFlow Lite Micro fue el siguiente:

1. Se calcula un valor de entrada x dentro del rango usado por el ejemplo.
2. El valor x se cuantiza utilizando la escala y el punto cero del tensor de entrada.
3. Se ejecuta la inferencia mediante el intérprete de TensorFlow Lite Micro.
4. La salida cuantizada se transforma nuevamente a punto flotante.
5. Se imprime la salida obtenida y el tiempo de inferencia.

4. Implementación directa en C

Para la implementación directa se utilizó la función `sinf(x)`, ya que el modelo *hello world* aproxima una función sinusoidal. Esta implementación no utiliza el intérprete de TensorFlow Lite Micro ni la estructura interna del modelo. Por lo tanto, evita el costo asociado a la cuantización, decuantización, manejo de tensores y ejecución general del runtime de TensorFlow Lite Micro.

La función implementada fue:

```
float inferencia_directa(float x) {  
    return sinf(x);  
}
```

Esta implementación permite comparar el costo de ejecutar la función objetivo directamente en C frente al costo de ejecutar una red neuronal pequeña mediante TensorFlow Lite Micro.

5. Medición de tiempo de inferencia

Para medir el tiempo de inferencia se utilizó la función `esp_timer_get_time()`, que entrega el tiempo en microsegundos. La medición se realizó antes y después de la inferencia en ambos casos.

Para TensorFlow Lite Micro se midió el tiempo de ejecución de:

```
TfLiteStatus invoke_status = interpreter->Invoke();
```

Para la implementación directa se midió el tiempo de ejecución de:

```
float y_directo = inferencia_directa(x);
```

La salida generada por el programa tuvo el siguiente formato:

```
x,y_tf_micro,y_directo,time_tf_us,time_direct_us  
0.000000,0.000000,0.000000,142,12  
0.314159,0.372770,0.309017,57,9  
0.628319,0.559154,0.587785,68,7  
...
```

6. Resultados de salida

En la Figura 2 se muestra la comparación entre la salida generada por TensorFlow Lite Micro y la salida generada por la implementación directa en C.

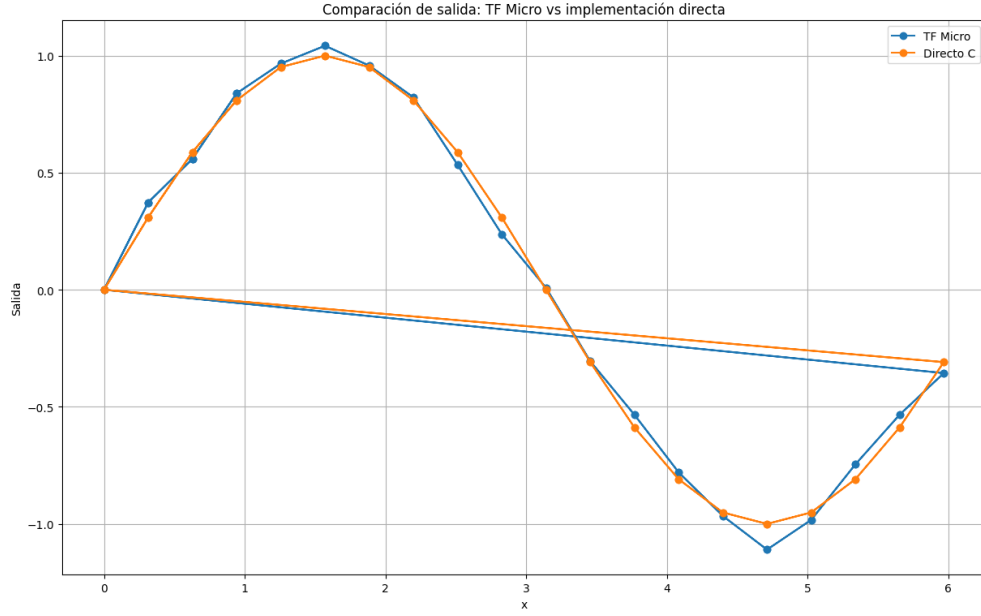


Figura 2: Comparación de salida entre TensorFlow Lite Micro y la implementación directa en C.

Se observa que ambas curvas presentan un comportamiento sinusoidal similar. La implementación directa corresponde a la función seno evaluada directamente, mientras que la salida de TensorFlow Lite Micro corresponde a una aproximación aprendida por el modelo. Por esta razón, las curvas no coinciden de manera exacta en todos los puntos.

Por ejemplo, para $x = 1,570796$, la implementación directa entrega un valor cercano a 1,0, mientras que TensorFlow Lite Micro entrega aproximadamente 1,042060. De forma similar, para $x = 4,712389$, la implementación directa entrega aproximadamente $-1,0$, mientras que TensorFlow Lite Micro entrega aproximadamente $-1,109837$. Estas diferencias son esperables, ya que el modelo no calcula la función seno de forma analítica, sino que la aproxima mediante una red neuronal entrenada y cuantizada.

A pesar de estas diferencias, la forma general de la señal se conserva. Esto muestra que el modelo de TensorFlow Lite Micro logra aproximar correctamente la tendencia de la función sinusoidal.

7. Resultados de tiempo de inferencia

En la Figura 3 se presenta la comparación del tiempo promedio de inferencia entre ambos métodos.

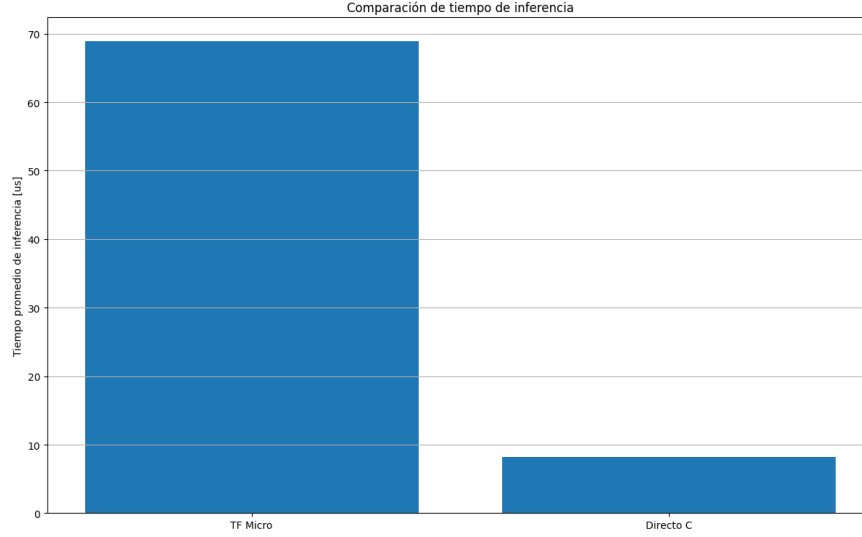


Figura 3: Comparación del tiempo promedio de inferencia entre TensorFlow Lite Micro y la implementación directa en C.

A partir de los datos obtenidos, el tiempo promedio de inferencia de TensorFlow Lite Micro fue cercano a:

$$T_{\text{TF Micro}} \approx 69 \mu s$$

Mientras que el tiempo promedio de la implementación directa fue cercano a:

$$T_{\text{Directo}} \approx 8 \mu s$$

Por lo tanto, la implementación directa fue aproximadamente:

$$\frac{T_{\text{TF Micro}}}{T_{\text{Directo}}} = \frac{69}{8} \approx 8,6$$

veces más rápida que la ejecución mediante TensorFlow Lite Micro.

Esta diferencia se explica porque TensorFlow Lite Micro debe realizar varias operaciones adicionales además del cálculo matemático del resultado. Entre ellas se encuentran la preparación del tensor de entrada, la cuantización del valor x , la ejecución del intérprete, la lectura del tensor de salida y la decuantización del resultado. En cambio, la implementación directa solamente evalúa una función matemática en C.

8. Análisis

Los resultados muestran que ambos métodos generan una salida con forma sinusoidal, lo que confirma que el modelo de TensorFlow Lite Micro está aproximando correctamente la función objetivo. Sin embargo, la implementación directa entrega valores más cercanos a la función seno ideal, ya que calcula directamente $\sin f(x)$. En cambio, TensorFlow Lite Micro entrega una aproximación obtenida desde una red neuronal entrenada y cuantizada, por lo que presenta pequeñas diferencias respecto al valor analítico.

En términos de tiempo de inferencia, TensorFlow Lite Micro presentó un tiempo promedio cercano a $69 \mu s$, mientras que la implementación directa presentó un tiempo promedio cercano a $8 \mu s$. Esto implica que la implementación directa fue aproximadamente 8.6 veces más rápida. La diferencia se debe principalmente a la sobrecarga del intérprete de TensorFlow Lite Micro, además de los pasos de cuantización, decuantización y manejo de tensores.

A pesar de que la implementación directa es más rápida para este caso particular, TensorFlow Lite Micro tiene una ventaja importante: permite desplegar modelos entrenados que pueden representar relaciones más complejas que una función matemática simple. En este ejercicio, como la función objetivo es conocida y sencilla, resulta más eficiente implementarla directamente. Sin embargo, en problemas más complejos, como detección de personas o clasificación de imágenes, una implementación manual sería mucho menos práctica.

Por lo tanto, este ejercicio permite observar el costo computacional de utilizar un framework de inferencia en un microcontrolador. TensorFlow Lite Micro entrega flexibilidad y portabilidad para ejecutar modelos de inteligencia artificial, pero introduce una sobrecarga mayor que una implementación directa optimizada para una tarea específica.

9. Conclusión

Se logró ejecutar el ejemplo *hello world* de TensorFlow Lite Micro sobre una ESP32-S3 y comparar su salida con una implementación directa en C. Ambas salidas presentaron una forma sinusoidal similar, aunque con pequeñas diferencias debido a que TensorFlow Lite Micro utiliza una red neuronal aproximada y cuantizada.

La implementación directa fue considerablemente más rápida, alcanzando un tiempo promedio cercano a $8 \mu s$, frente a aproximadamente $69 \mu s$ para TensorFlow Lite Micro. Esto muestra que, cuando la función a calcular es conocida y simple, una implementación directa puede ser más eficiente. Sin embargo, TensorFlow Lite Micro sigue siendo útil cuando se requiere ejecutar modelos entrenados para tareas más complejas, donde una implementación manual no sería práctica.

10. Anexos

10.1. Salida serial utilizada

```
x,y_tf_micro,y_directo,time_tf_us,time_direct_us
0.000000,0.000000,0.000000,142,12
0.314159,0.372770,0.309017,57,9
```

0.628319,0.559154,0.587785,68,7
0.942478,0.838731,0.809017,65,22
1.256637,0.965812,0.951057,76,7
1.570796,1.042060,1.000000,65,3
1.884956,0.957340,0.951056,70,10
2.199115,0.821787,0.809017,71,6
2.513274,0.533738,0.587785,69,17
2.827433,0.237217,0.309017,65,9
3.141593,0.008472,-0.000000,72,13
3.455752,-0.304993,-0.309017,64,7
3.769912,-0.533738,-0.587786,70,10
4.084070,-0.779427,-0.809017,67,8
4.398230,-0.965812,-0.951056,68,5
4.712389,-1.109837,-1.000000,65,6
5.026548,-0.982756,-0.951056,72,8
5.340708,-0.745539,-0.809017,64,4
5.654867,-0.533738,-0.587785,70,10
5.969026,-0.355825,-0.309017,67,9